SPOTIFY: ARCHITECTURE OF THE APP

Distributed Computing | University of Hyderabad

Student	K. Amulya	**Reg. No.**	23MCCE22
App studied	Spotify (music streaming service)	**Developed app**	SimpleStream (Spotify-like app)
Programme	Integrated M.Tech CSE	**Date**	October 2026

1. Introduction and Scope

This report summarises what I learned about the design and architecture of **Spotify**, the music streaming service, and compares it with the app I built for this course: **SimpleStream**, a pure-Java, socket-based Spotify-like application with a website, a terminal client and a 3-node backend. The study uses first-party sources wherever possible: Spotify Engineering blog posts, the research paper by two Spotify engineers, Spotify for Developers documentation

2. The Original Spotify – Overview and Version History

Spotify launched in 2008 and now offers a catalogue of tens of millions of songs; its 2015 engineering post cites over 30 million songs and over 1.5 billion user-created playlists. The service has no single public version number; the sources show it evolving through three architectural stages, listed below.

Period	Stage	What changed (source)
2008 – 2014	P2P-assisted desktop	Desktop clients streamed from Spotify servers and from other users' clients over a peer-to-peer network; mobile clients used servers only. In April 2014 the P2P network was shut down because Spotify had enough servers worldwide
2014 – 2017	Containers and Helios	Containerised microservices ran on Helios, Spotify's own open-source orchestrator, from 2014. Event delivery used Kafka and Hadoop and carried over 700,000 events per second
2015 – 2018	Move to Google Cloud	Cloud strategy explored in early 2015; all migration sprints complete and traffic fully on Google Cloud in May 2017; first of four data centres closed in December 2017, the rest in 2018. Event delivery rebuilt on Pub/Sub and Dataflow.
2018 onward	Kubernetes and platform	Kubernetes adopted alongside Helios from late 2018; thousands of microservices, one service above 10 million requests per second Backstage developer portal, now open source .
2017 onward	Public developer platform	Player API launched in 2017 as the Connect Web API ; Web API, Web Playback SDK and Connect device SDK for outside developers.

Version summary. Spotify is versioned by architectural stage (P2P client–server → own data centres with containers → Google Cloud with Kubernetes) and by its public APIs. My app is versioned by the build in Section 5.

3. Architecture of the Original Spotify

3.1 Layered view (interpreted from official sources)

- **Client layer:** desktop, mobile and web players, plus Spotify Connect devices such as speakers. A phone can act as a remote control while another device plays the audio
- **Entry and API layer:** each client keeps one long-lived, multiplexed TCP connection to a Spotify server [1]; outside developers use the Web API with OAuth access tokens and scopes
- **Service layer:** thousands of independently owned microservices in containers, orchestrated by Helios and later Kubernetes, using gRPC and Envoy; each owned by a small autonomous squad
- **Data layer:** Cassandra clusters for user profiles and entity metadata, replicated between data centres; a separate event pipeline (Pub/Sub, Dataflow, Avro on HDFS and Cloud Storage) for analytics.
- **Platform layer:** own data centres in four regions until 2018, then Google Cloud; Backstage as the developer portal.

3.2 How music reaches the player (P2P-era design)

The most detailed public description of Spotify's streaming path is the 2010 paper by Kreitz and Niemelä [1], summarised in Figure 1. The client first checks its encrypted local cache (LRU, default 10% of free disk). For a track it does not hold, it streams from the server over its open connection and, on desktop, also asks a **tracker** (which remembers the 20 most recent peers per track) and its neighbours for peers. Files are split into **16 kB chunks**. For tracks played in sequence (about 61% of plays) the client starts looking for the next track when 30 seconds remain and prefetches its start from the server at 10 seconds remaining. Measured median start-up latency was **265 ms**, and data came **55.4% from cache, 35.8% from peers and 8.8% from servers**

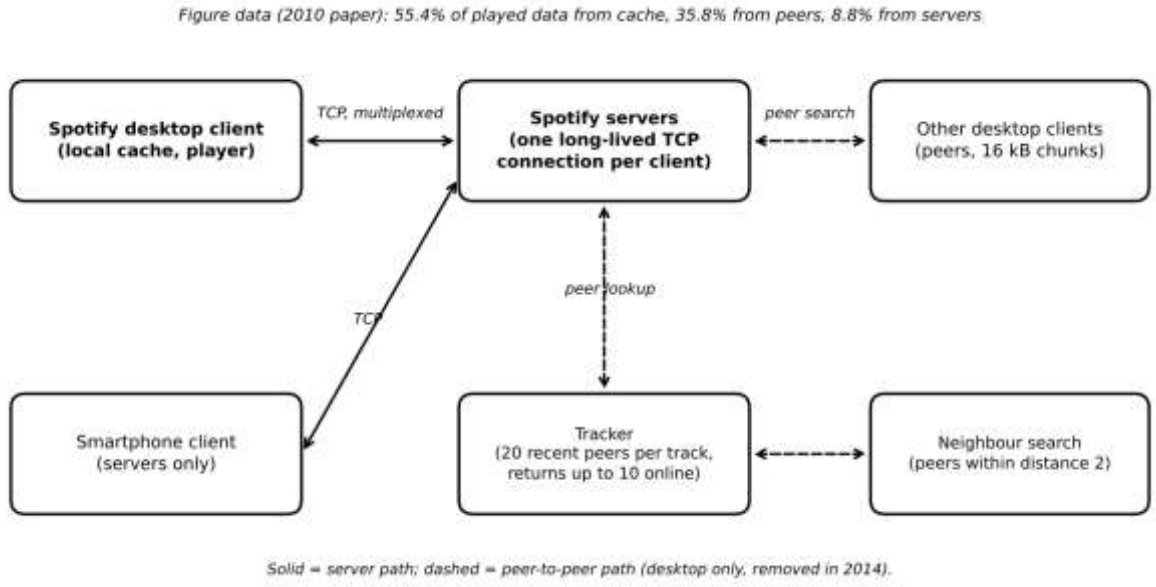


Figure 1. Spotify streaming path in the P2P era (drawn from the 2010 paper [1]).

3.3 Backend platform, data and events

- **Cloud migration**: about 2,000 services, 20,000 daily data-pipeline runs and over 100 petabytes of data across about 100 teams were moved; each team had a two-week migration sprint; Spotify chose to be "fundamentally in the music business", not to run data centres.
- **Kubernetes**: new service creation fell from up to an hour to seconds or minutes, and average CPU utilisation improved two to three times.
- **Event delivery** : the old Kafka/Hadoop pipeline had Hadoop as a single point of failure and tightly coupled components; the rewrite uses one Pub/Sub topic per event type (about 1,000 event types) and Dataflow jobs writing hourly Avro buckets.
- **Cassandra**: chosen for horizontal scaling, cross-site replication and low latency in preference to strict consistency; doubling the cluster roughly halved median and 99th-percentile latency.

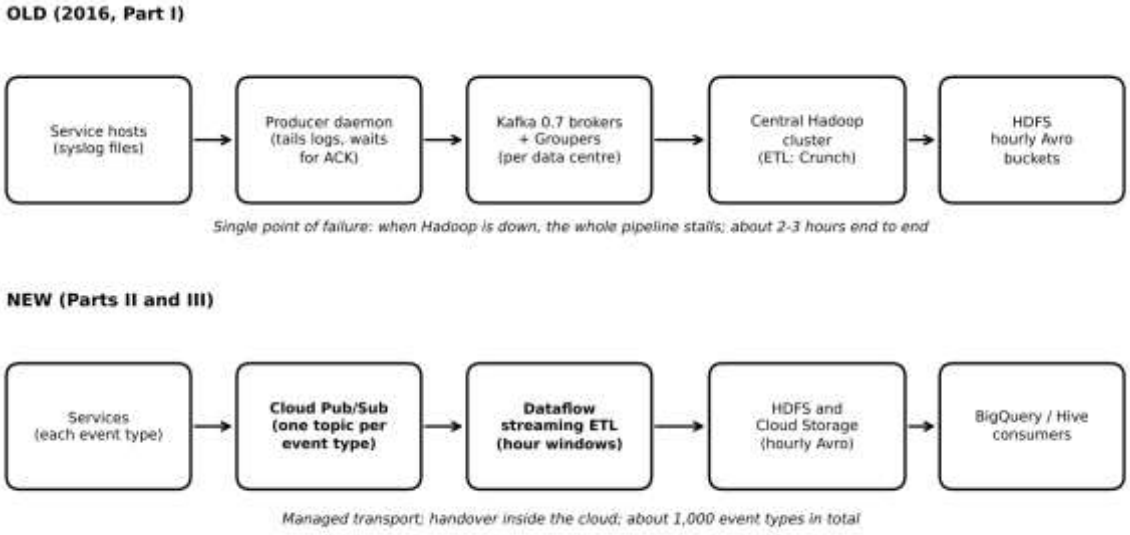


Figure 5. Spotify event delivery: old Kafka/Hadoop pipeline vs new Pub/Sub/Dataflow design (from [11][12]).

3.4 Playback control, audio quality and access

Item	Original Spotify	Developed app (SimpleStream)
Playback control	Spotify Connect: the phone is the remote, the speaker plays; only the active device plays.Player API starts, pauses, seeks and queues playback with an OAuth token.	No remote control; the player bar plays in the browser and the terminal plays on its own machine.
Developer access	Web API: albums, artists, tracks, playlists, search, library, player and users ; Web Playback SDK makes a browser a Connect device [9]; most player commands need Premium	Internal `/api/*` routes used only by its own website; no public API.
Audio format	Ogg Vorbis about 160 / 320 kbit/s in 2010; today AAC (web 128 / 256 kbit/s) and up to lossless FLAC, with an Automatic level that follows the network	One quality: WAV generated by `AudioGenerator` (or the user's own mp3).
Plans	Free and Premium; Very high and Lossless are Premium only	No plans; any signed-up user can use everything.
Clients	Desktop, mobile, web, Connect hardware	Website and terminal client.

3.5 How Spotify organises its engineers

Spotify's white paper explains how its teams are organised, which shapes the microservice architecture: each squad owns services end to end, and shared knowledge moves sideways through chapters and guilds.

Unit	Definition (source [15])	Developed app
Squad	Small, cross-functional, self-organising team with a long-term mission; works like a mini-startup	Single developer
Tribe	Group of squads in related areas, kept under about 100 people	Not applicable
Chapter	People with similar skills across squads in one tribe; the chapter lead is the line manager	Not applicable
Guild	Voluntary organisation-wide community of interest	Not applicable
Backstage	Developer portal with a Software Catalog, Software Templates and TechDocs, open sourced under the CNCF	`README.md` documents the code

4. Architecture of the Developed App

SimpleStream is a distributed system written with the JDK only [16]. The browser talks to a small HTTP server (`SpotifyWeb`, port 8080); the terminal client talks to the backend directly. Both reach the backend through

`Balancer`, which sends each request to one of three identical `MusicServer` nodes (ports 5001–5003). Every hop is a raw `java.net.Socket`, as in Figure 2.

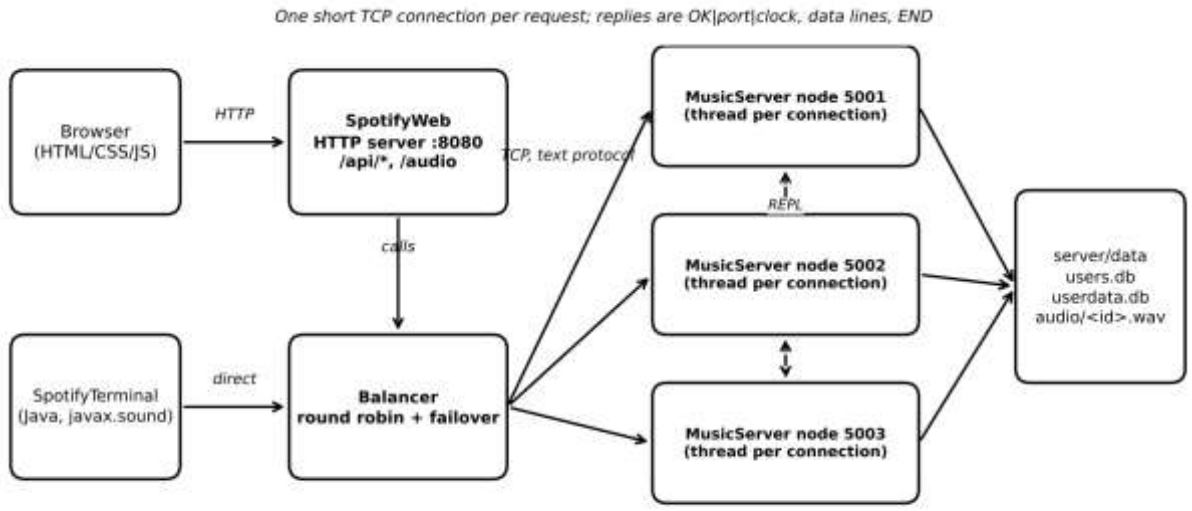


Figure 2. Architecture of the developed app (clients, gateway, balancer, three nodes, data files).

4.1 Components and responsibilities

Component	Tier	Responsibility
`SpotifyWeb`	Gateway	Hand-written HTTP server: serves `public/`, turns `/api/*` into a socket call, and streams `/audio/<id>` with HTTP Range (206) so the browser can seek.
`SpotifyTerminal`	Client	Command-line client; plays WAV with `javax.sound`; prints the serving node and the logical clock.
`Balancer`	Client side	Round-robin choice of node; failover to the next node (800 ms connect, 4 s read timeouts).
`MusicServer` (×3)	Application	Accounts, likes, playlists, history, play counts; text protocol; thread per connection; write-all replication; Lamport clock.
`users.db`, `userdata.db`	Data	Text files saved after every write by a temporary file and an atomic `Files.move`; reloaded at start-up.
`Catalog`, `AudioGenerator`	Content	18 songs (6 Hindi, 6 English, 6 Telugu); a different WAV tune is generated for each.

4.2 Distributed paradigms and their counterpart in the original

Paradigm	Where it appears in SimpleStream	Counterpart in the original Spotify
Client–server	Website and terminal → `MusicServer` nodes	Clients keep one TCP connection to Spotify servers
Peer-to-peer	Not used	Used on desktop until 2014
Load balancing	`Balancer.send()` round robin	Kubernetes and the service layer
Replication	Write-all: `REPL` to the other two nodes	Cassandra replicated
Fault tolerance	Failover; “Crash last node” button	Remove single points of failure, as in the event rewrite
Logical time	Lamport clock: `max(own, received) + 1`	Not described in the sources studied
Stateless requests	NEXT/PREV carry song and playlist	Kubernetes migration started with stateless services
Persistence	Atomic file save after every write	Cassandra and cloud storage
Event-driven design	Not used; direct calls	Pub/Sub topic per event type

Life of a write. A write (`SIGNUP`, `LIKE`, `PLCREATE`, `PLDELETE`, `PLADD`, `PLREMOVE`, `PLAY`) is applied on the node that receives it, which increments its clock, saves to disk and forwards `REPL<TAB>clock<TAB>command` to the other nodes. Each replica sets `clock = max(own, received) + 1`. The reply `OK|port|clock` tells the client which node answered (Figure 3). If a node does not answer, `Balancer` tries the next one.

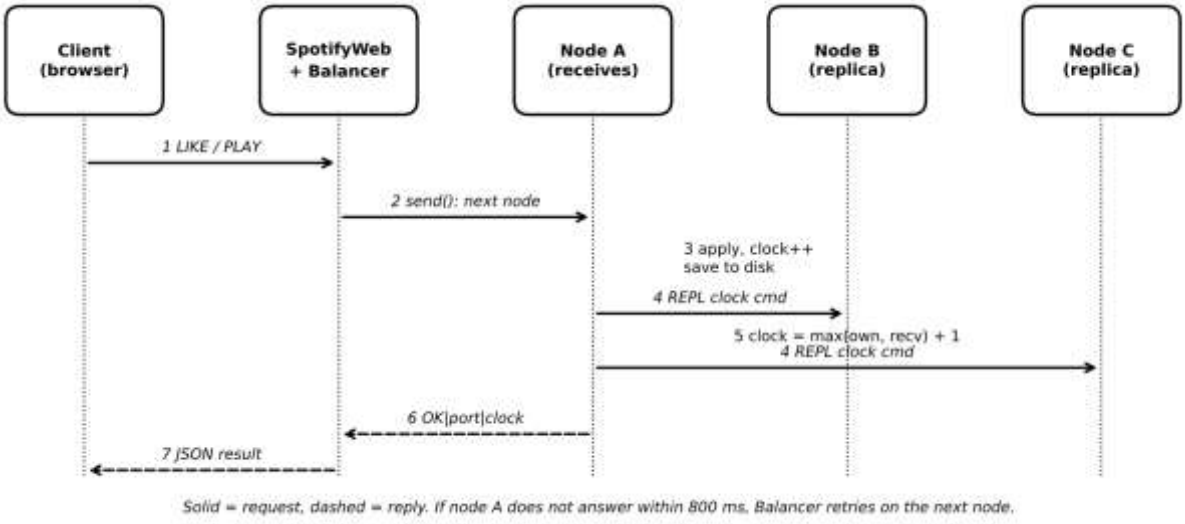


Figure 3. Life of a write request in the developed app (solid = request, dashed = reply).

5. Version Information

	Original Spotify	**Developed app (SimpleStream)**
Versioning scheme	Architectural stages and public APIs; no single backend version number	One code base; the package analysed is "SimpleStream_Final". It carries no version number, so it is called **v1.0 (final build)** in this report.
Current version	Google Cloud + Kubernetes platform; Web API, Player API, Web Playback SDK	v1.0: website + terminal client, 3-node backend, email login, likes, playlists, history, play counts, persistence.
Stage 1	2008–2014: client–server with peer-assisted desktop streaming	Client–server with sockets: `MusicServer`, `Catalog`, accounts.
Stage 2	2014–2017: Docker containers on Helios	3 nodes with `Balancer`, round robin and failover.
Stage 3	2017–2018: Google Cloud, Pub/Sub and Dataflow events	Write-all replication, Lamport clock and "Crash last node" demo.
Stage 4	2018 onward: Kubernetes, Backstage, public APIs	Persistence to `users.db` / `userdata.db`; email login on the website.

6. Comparison – Original vs. Developed App

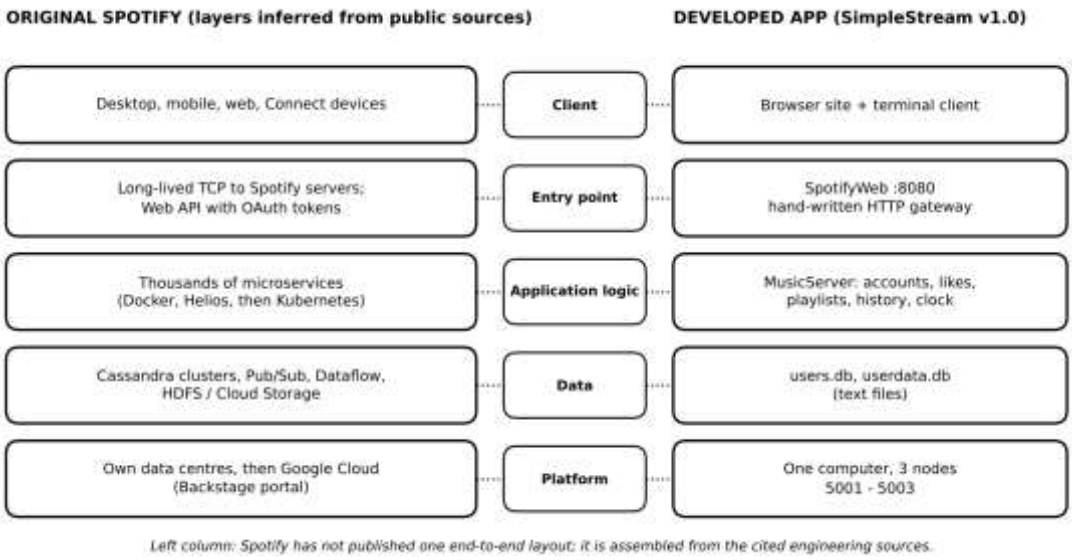


Figure 4. Layer-by-layer mapping between the original Spotify and the developed app.

6.1 Functional comparison

Feature	Original Spotify	Developed app
Catalogue	Over 30 million songs	18 songs (6 Hindi, 6 English, 6 Telugu)
Search	Web API Search resource	Search by title, artist or language
Library / likes	Library resource	Heart on a song; per-user Library
Playlists	Over 1.5 billion created ; Playlists resource	Create, open, add, remove, delete
Recently / most played	Personalisation from user-profile data	Recently played and play counts on Home
Seek and next/previous	Player API: seek, queue	Seek bar, Next, Previous (stateless)
Own music	Local files (not covered in the sources)	Drop `<id>.mp3` into `server/data/audio/`
Accounts	Spotify accounts, Free / Premium	Email, username, password; SHA-256 hash

6.2 Architectural and distributed-systems comparison

Aspect	Original Spotify	Developed app
Scale	Global; thousands of microservices; one service above 10 million requests/s	One computer; 3 nodes on localhost
Communication	Long-lived multiplexed TCP; gRPC and Envoy	Short TCP connection per request; text protocol
Load balancing	Kubernetes and service layer	Explicit, readable `Balancer.send()`
State and storage	Cassandra clusters; HDFS / Cloud Storage	Two text files shared by the nodes
Replication	Cross-site; latency preferred over strict consistency	Write-all; a restarted node is not re-synced
Caching / prefetch	Local cache, prefetch of next track	None beyond the browser
Security	OAuth tokens with scopes [14]; encrypted cache	Unsalted SHA-256; username sent per request
Deployment	Docker, Helios, Kubernetes; staging per pull request	`run.bat` / `run.sh`; two commands
Observability	Dashboards and alerts; completeness checks	Sidebar node status, console logs, clock display
Organisation	Squads, tribes, chapters, guilds; Backstage	Single developer, one code base

6.3 One action end to end: playing a song

Step	Original Spotify (interpreted)	Developed app (exact)
1	Client sends the play request on its open connection.	Browser calls `/api/play` on `SpotifyWeb`.
2	Services handle it; the track is checked in the local cache first	`Balancer.send()` picks the next node in round robin.
3	If missing, audio is fetched from servers (and peers, in the P2P era) in 16 kB chunks [	Browser requests `/audio/<id>.wav`; `SpotifyWeb` answers 206 for the byte range.
4	The next track is prefetched near the end	Next song requested only when the user presses Next or the song ends.
5	Play event goes to the event pipeline (Pub/Sub, Dataflow)	`PLAY` is a write: history and play count updated, `REPL` sent to two nodes.
6	User data is read and written in replicated Cassandra	Node saves `userdata.db` atomically; reply `OK|port|clock`.

7. Summary of my findings

- **The core pattern is the same.** Both systems are client–server designs that spread work over many servers, balance load, replicate user data and persist it. Building SimpleStream with sockets made every step visible: the wire format, the thread per connection and the replication call.
- **The difference is scale and operations.** Spotify's hard problems are running thousands of services, moving data across regions and shipping safely; its engineers moved from their own data centres to the cloud so they could work "up the stack" [3]. SimpleStream's hard part is the coordination logic.
- **Architecture follows history.** Spotify dropped P2P when it had enough servers [2] and replaced Helios with Kubernetes when a larger community offered more [4]. Designs are right for a scale and a time.
- **Measure and prefetch.** The 2010 paper reports exact latency and data-source numbers and shows prefetching cut median latency from 390 ms to 265 ms [1]. SimpleStream has no such measurements.
- **Loose coupling prevents total failure.** The event-delivery rewrite removed a single point of failure [11], and the squad model keeps teams independent [15]. SimpleStream's nodes are independent, but a crashed node cannot rejoin.

8. Conclusion

Studying Spotify's engineering material showed a service that grew from a client–server design with peer-assisted streaming into thousands of containerised microservices on Google Cloud, with Cassandra for user data, Pub/Sub and Dataflow for events and a public developer platform on top [1][3][4][8][12][13]. SimpleStream reproduces the shape of such a system in miniature and exposes its distributed mechanics (client–server, round-robin balancing, write-all replication, failover, Lamport clock, persistence), while deliberately leaving out what a course project cannot rebuild: Spotify's scale, caching and prefetching, adaptive audio, managed infrastructure and engineering platform.

9. References

- Backend infrastructure at Spotify — Spotify Engineering (2013)
- Building the Future of Our Desktop Apps — Spotify Engineering (2021)
- Spotify's Event Delivery – The Road to the Cloud, Part I — Spotify Engineering (2016)
- Spotify's Event Delivery – The Road to the Cloud, Part II — Spotify Engineering (2016)
- How We Use Backstage at Spotify — Spotify Engineering (2020)
- Achieving Fault-Tolerance with Intelligent Daemons — Spotify Engineering (2013)
- Helios (Spotify's open-source container orchestrator) — Spotify GitHub
- Spotify's journey to cloud: Kafka to Google Cloud Pub/Sub — Google Cloud Blog (2016)

- **23MCCE22 (K.AMULYA)**